App Dev Project Report

1. Student Details

Name: Shakthi Kumaran Gnanavel
Roll Number: 23f2003594
Email: 23f2003594@ds.study.iitm.ac.in
About Me: I am a student currently pursuing Diploma in the IIT Madras BS Degree Program with an interest in machine learning and web development , particularly in designing and building scalable , user-centric applications . I enjoy working across the full stack , integrating frontend and backend systems to create efficient and reliable solutions .

---

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To design and develop a web-based placement management system that enables students to apply for job opportunities, companies to manage recruitment processes, and administrators to oversee and control the entire placement workflow efficiently.

Approach:

The application was developed in a backend-first manner using Flask with a modular structure for admin, company, and student functionalities. Initially, core features such as authentication, database operations, job postings, and application workflows were implemented and tested using simple skeleton HTML templates. Once the backend logic and data flow were fully validated, focus shifted to building the frontend, improving user interaction, navigation, and presentation. This approach ensured a stable and functional system before enhancing the user interface.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist primarily with debugging errors, refining SQL queries, structuring backend logic, and generating API documentation in YAML format. Additionally, I

used **Gemini AI 3** for assistance with frontend styling, particularly for improving CSS and layout design. These tools were helpful in enhancing code clarity and handling edge cases during development.

The overall extent of AI/LLM usage is estimated to be around 15–18%, mainly limited to suggestions, debugging assistance, documentation support, and minor frontend styling improvements. All core implementation, integration of modules, and final decision-making were carried out manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
HTML5/CSS	Standard web technologies used for structuring and styling the portal.
Werkzeug	Security library used for password hashing and session management.
Python	Core programming language used for backend development
JavaScript	Basic client-side interactivity (form handling,UI behaviour)
SQLite	Lightweight local database for storing user data
Git & GitHub	Version control and project management

---

5. Database Schema / ER Diagram

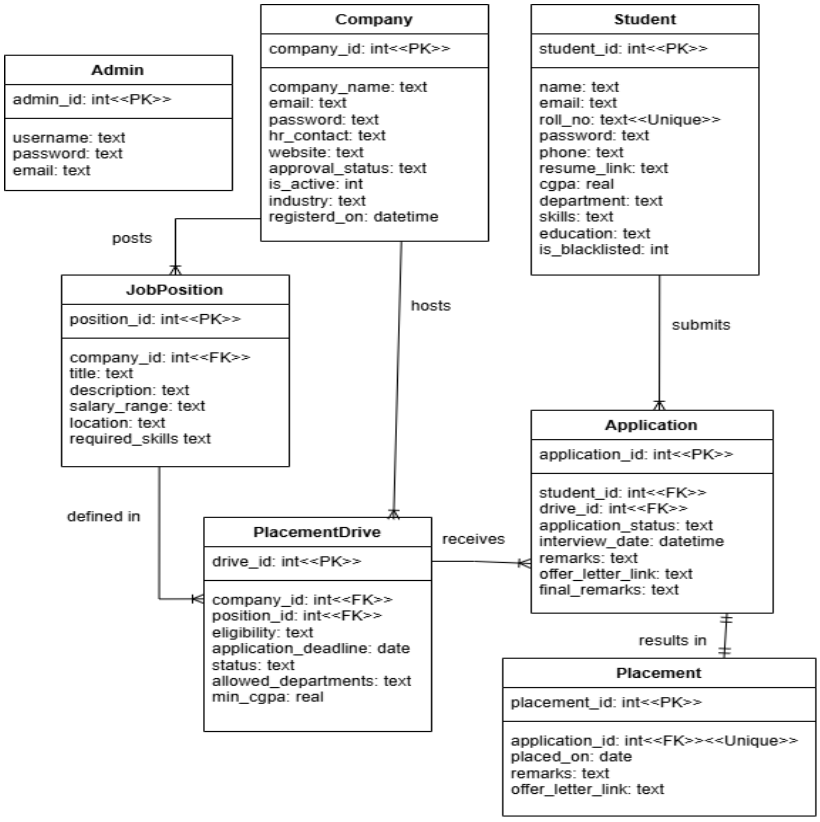
Tables:

1. **admin** — stores portal administrators for managing companies and students. (admin_id, username, password, email)

2. **company** — stores company profiles and their approval/active status. (company_id, company_name, email, password, hr_contact, website, approval_status, is_active, registered_on, industry)
3. **student** — stores candidate academic profiles, skills, and contact info. (student_id, name, email, password, phone, resume_link, cgpa, graduation_year, is_active, is_blacklisted, registered_on, roll_no, education, skills, department)
4. **job_position —** contains specific role details created by companies. (position_id, company_id, title, description, salary_range, location, job_type, required_skills)
5. **placement_drive —** represents a recruitment event linking a job to a deadline. (drive_id, company_id, position_id, eligibility, application_deadline, status, allowed_departments, min_cgpa)
6. **application —** tracks student's applications per drive. (application_id, student_id, drive_id, application_status, interview_date, remarks, offer_letter_link, final_remarks)
7. **placement —** stores the final record of students who accepted an offer. (placement_id, application_id, placed_on, remarks, offer_letter_link)

Relationships:

- One-to-One → application → placement
- One-to-Many → company → job_position
- One-to-Many → company → placement_drive
- One-to-Many → job_position → placement_drive
- One-to-Many → student → application
- One-to-Many → placement_drive → application
- Many-to-One → job_position → company
- Many-to-One → placement_drive → company
- Many-to-One → application → student
- Many-to-One → application → placement_drive



---

6. API Resource Endpoints - Core

Endpoint	Method	Description
`/login`	POST	Authenticates user and initializes role-based session.
`/logout`	GET	Terminates session and clears user data
`/register/student`	POST	Creates a new student profile in the database.
`/register/company`	POST	Submits company details for Admin verification.
`/student/dashboard`	GET	Displays application status and interview alerts.
`/student/drives`	GET	Fetches job drives based on student eligibility.

`/student/apply{id}`	POST	Processes a new job application submission.
`/student/profile`	POST	Updates student academic and professional details.
`/student/accept_offer/{id}`	POST	Finalizes a job offer and records the placement.
`/company/dashboard`	GET	Provides recruiter-specific analytics and drive counts.
`/student/reject_offer/{id}`	POST	Formally declines a pending job offer.
`/company/drive/create`	POST	Submits a new job position and recruitment drive.
`/company/schedule_interview/{id}`	POST	Sets interview logistics and candidate instructions.
`/company/select/{id}`	POST	Submits a offer link for a selected candidate.
`/company/drive/close{id}`	POST	Manually terminates an active recruitment drive.
`/admin/dashboard`	GET	Overview of placement_portal statistics.
`/admin/company/approve/{id}`	POST	Grants a company permission to post recruitment drives.
`/admin/student/toggle/blacklist/{id}`	GET	Toggles a student's access to the placement portal.
`/admin/drive/approve/{id}`	POST	Activates a pending drive for student applications.
`/admin/placements`	GET	Generates the final report of all successful hires.
`/admin/applications`	GET	Provides a filterable list of all portal applications.
`/admin/student/{id}`	GET	Allows admin to view detailed student profile .

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1MqHDOnXCJnY3Z_QJv2ECebbq06joUGQr/view?usp=sharing

---